

TAREA ENTREGABLE A

Documentación

Prueba de Cross Site Scripting (XSS) en el “foro vulnerable”

1. **Acción realizada:** Introduje el payload `<script>alert('XSS')</script>` en el campo de comentarios del formulario de creación de temas. Me aseguré de probar el envío en distintos navegadores, como Chrome y Firefox, para confirmar que el comportamiento era consistente.
2. **Resultado obtenido:** La ejecución del script fue exitosa, y se mostró una alerta con el texto XSS al cargar la página del foro. Este comportamiento confirma que el foro es vulnerable a ataques XSS debido a la falta de validación o sanitización de los datos de entrada.

Código fuente implicado:

```
<?php
if (isset($_POST['comentario'])) {
    $comentario = $_POST['comentario'];
    $query = "INSERT INTO comentarios (texto) VALUES
('$comentario')";
    mysqli_query($conexion, $query);
}
?>
```

3. **Observación personal:** Durante la prueba, noté que si el script contenía caracteres especiales (como comillas simples o dobles), el sistema podía mostrar errores SQL, lo que indica que hay múltiples vectores de ataque posibles.

Prueba de inyección SQL en el “foro vulnerable”

1. **Acción realizada:** En el campo de búsqueda de temas del foro, introduje el payload `' OR '1'='1` para intentar acceder a todos los registros de la base de datos sin restricciones.
2. **Resultado obtenido:** El sistema mostró todos los temas almacenados, incluso aquellos que deberían estar restringidos. Esto confirma que la consulta SQL no valida correctamente los datos de entrada.

Código fuente implicado:

```
<?php
```

```
$busqueda = $_GET['query'];  
$query = "SELECT * FROM temas WHERE titulo LIKE  
'%$busqueda%'";  
$result = mysqli_query($conexion, $query);  
?>
```

3. **Observación personal:** Inicialmente, olvidé codificar las comillas simples en el payload, lo que provocó errores SQL. Al corregirlo, la prueba se ejecutó correctamente.
-

Prueba de extracción de datos mediante SQLi en DVWA

1. **Acción realizada:** Utilicé el payload ' UNION SELECT 1, username, password FROM users -- en el campo de búsqueda para intentar acceder a las credenciales de usuario.
2. **Resultado obtenido:** Se mostraron los nombres de usuario y contraseñas almacenados en la base de datos, lo que confirma una vulnerabilidad SQLi clásica.

Código fuente implicado:

```
$query = "SELECT id, name FROM products WHERE name LIKE  
'%$input%'";
```

- 3.
4. **Observación personal:** DVWA simplifica las pruebas al mostrar el resultado directamente en pantalla, pero sería interesante verificar este comportamiento en un entorno más realista.

Explicación de la Cuestión 1

La cuestión requería discutir el impacto y mitigación de las vulnerabilidades:

- **Impacto:** XSS permite robar cookies o redirigir usuarios. SQLi puede exponer datos confidenciales y comprometer sistemas.
- **Mitigación:**
 - Escapar entradas de usuario.
 - Utilizar parámetros preparados (`mysqli_stmt` o PDO).
 - Implementar validaciones estrictas en todos los inputs.

TAREA ENTREGABLE B

Documentación

Pruebas repetidas sobre el foro vulnerable

1. **Prueba de Cross Site Scripting (XSS):** Durante esta prueba, repetí el uso del payload `<script>alert('XSS')</script>` y confirmé nuevamente que se ejecuta correctamente en diferentes navegadores.
2. **Prueba de inyección SQL:** Verifiqué que el payload `' OR '1'='1` continúa mostrando todos los registros de la base de datos. Adicionalmente, probé con payloads más complejos para intentar filtrar información específica.

Muestras de logs de ModSecurity:

1.Alerta generada en /var/log/apache2/error.log:

```
[error] [client 192.168.1.100] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator `Rx' with variable `ARGS:comentario'" at ARGS:comentario.
```

2.Alerta generada en /var/log/apache2/modsec_audit.log:

```
--da39a3ee5e6b4b0d3255bfef95601890afd80709--  
[hostname "modsecurity.ssi.net"] [uri "/foro/comentarios"]  
[unique_id "Y2MkFFfR"]  
Rule ID: 12345
```

Regla de ModSecurity identificada:

```
SecRule ARGS ".*<script>.*"  
"id:12345,phase:2,deny,status:403,msg:'XSS Attack' "
```

Explicación: Esta regla bloquea cualquier solicitud que contenga etiquetas `<script>` en los argumentos HTTP, evitando intentos de XSS.

TAREA ENTREGABLE C

Documentación

Configuración de ModSecurity en modo rechazo

Cambios realizados: Edité el archivo `/etc/modsecurity/modsecurity.conf` y cambié el valor del parámetro `SecRuleEngine` a `On`. Después, reinicié Apache con el comando:

```
systemctl restart apache2
```

1. **Cambios realizados:** Edité el archivo `/etc/modsecurity/modsecurity.conf` y cambié el valor del parámetro `SecRuleEngine` a `On`. Después, reinicié Apache con el comando:

```
systemctl restart apache2
```

2. **Pruebas realizadas:**

- Payload XSS: `<script>alert('XSS')</script>` → Generó un error 403 en el navegador.
- Payload SQLi: `' OR '1'='1` → También generó un error 403, confirmando que las reglas de rechazo estaban activas.

3. **Observación personal:** Durante las pruebas, me aseguré de acceder al foro utilizando el nombre de la máquina

<http://modsecurity.ssi.net/foro>. Inicialmente, había utilizado la dirección IP, lo que provocó que las reglas no se activaran.

Resultados obtenidos: El servidor bloqueó todas las solicitudes maliciosas y registró las alertas correspondientes en los logs. Esto demuestra que ModSecurity en modo rechazo es efectivo para mitigar ataques XSS y SQLi.